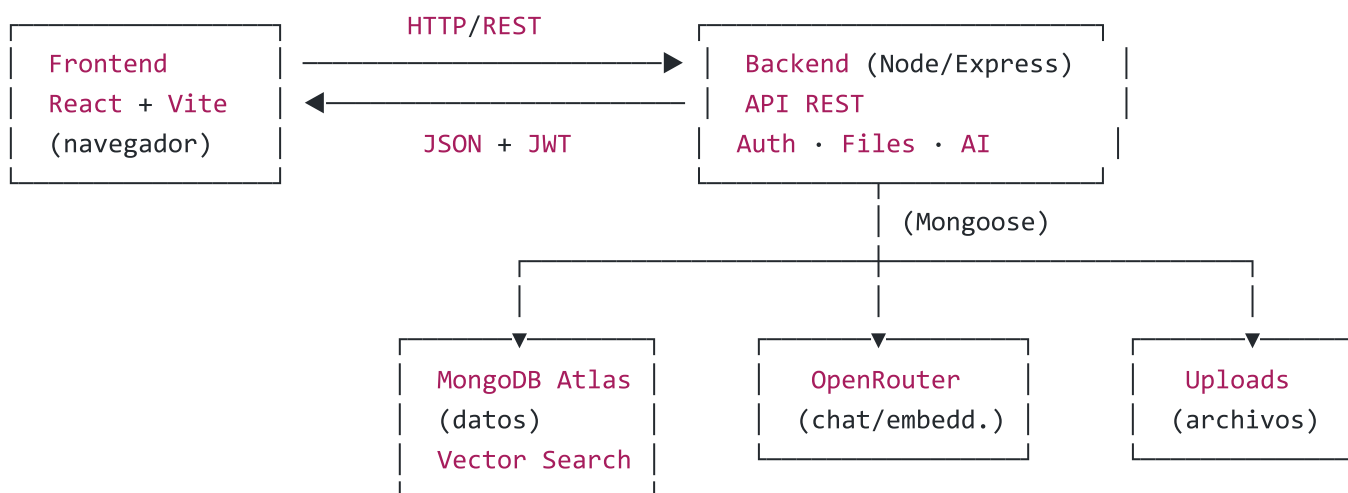


Documento 02 — Diseño del Sistema

Proyecto: Sistema Inteligente de Gestión y Análisis Documental **Versión:** 1.0 **Fecha:** 2 de septiembre de 2026 **Estado:** Borrador final

1. Arquitectura general

La solución sigue una arquitectura **cliente-servidor** con separación de frontend y backend, base de datos en la nube (MongoDB Atlas) y consumo de servicios de IA externos.



- **Cliente:** aplicación SPA en React + Vite. Se comunica con el backend mediante REST.
- **Backend:** API REST en Node.js + Express, organizada por capas.
- **Base de datos:** MongoDB Atlas guarda datos (usuarios, repositorios, documentos, logs) y los vectores (embeddings). MongoDB Atlas **Vector Search** permite búsqueda semántica sin levantar infraestructura adicional.
- **IA externa:** OpenRouter (APIs compatibles con OpenAI). Se usan dos modelos:
 - Chat/completions → clasificación, resumen, extracción de campos y respuestas RAG.
 - Embeddings → representación vectorial de fragmentos de texto.
- **Almacenamiento de archivos:** el sistema de archivos local del backend (backend/uploads), con servido estático para descarga. (Opción de mejora futura: S3/GridFS.)

Justificación de Atlas Vector Search sobre una BD vectorial aparte

- **Un solo motor de datos:** documentos y vectores viven en la misma colección/instancia; no hay sincronización entre sistemas.
- **Costo cero adicional:** ya se paga el cluster de Atlas; no se agrega un servicio externo.

- **Menos infraestructura:** no es necesario orquestar ni mantener una base vectorial separada (ej. Pinecone, Qdrant, Weaviate) para el volumen actual.
- **Simplicidad operativa:** los índices vectoriales se crean desde la consola de Atlas sin código de administración adicional.
- **Escalable si crece el volumen** sin cambiar la arquitectura.

2. Arquitectura por capas

2.1 Frontend (React)

```
frontend/
  src/
    api/          -> cliente HTTP (axios/fetch) hacia el backend
    context/      -> AuthContext (estado del usuario y token)
    components/   -> componentes reutilizables (UI)
    pages/        -> páginas (Login, Register, Dashboard, Repos, FileDetail, Search, Chat)
    routes/       -> rutas protegidas con React Router
    utils/        -> helpers, formateo de fechas, etc.
```

Estado global: AuthContext guarda user y token ; el token se persiste en localStorage para mantener la sesión al recargar.

2.2 Backend (Express)

```
backend/src/
  config/        -> env.js (configuración), db.js (MongoDB)
  models/        -> schemas Mongoose (User, Repository, Document, ProcessingLog)
  controllers/   -> lógica de negocio por recurso (auth, repository, file, ai)
  routes/        -> definición de endpoints REST
  middleware/     -> authMiddleware, authorize, uploadMiddleware, errorMiddleware
  services/      -> aiService (OpenRouter), extractionService (PDF/DOCX/TXT), embeddingService
  utils/         -> generateToken, respuestas helpers
  server.js      -> punto de entrada
```

Flujo de una petición: **ruta** valida la forma → **middleware** autentica/autoriza → **controlador** aplica reglas → **servicio** ejecuta lógica de IA/texto → **modelo** persiste en MongoDB.

2.3 Datos (MongoDB)

Colecciones: users , repositories , documents , processinglogs .

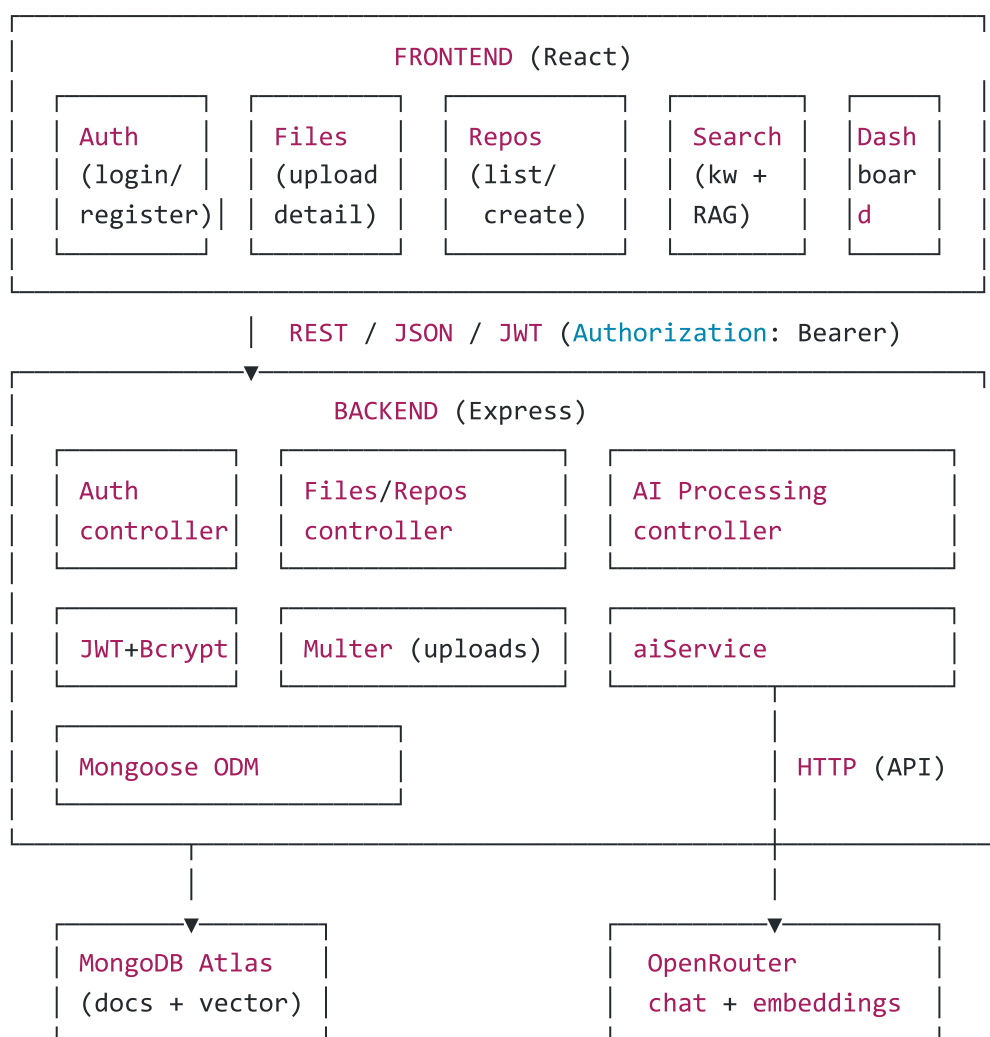
2.4 IA (servicios)

```

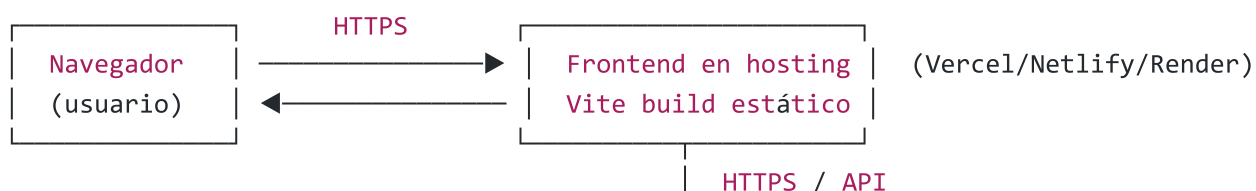
aiService
├ classifyDocument(text)      -> categoría (Contrato/Factura/Reporte/Otro)
├ summarizeDocument(text)    -> resumen de 3-5 líneas
├ extractKeyInfo(text, tipo) -> campos clave según categoría
├ answerWithContext(question, contextChunks) -> respuesta RAG
└ generateEmbedding(text/chunk) -> vector numérico

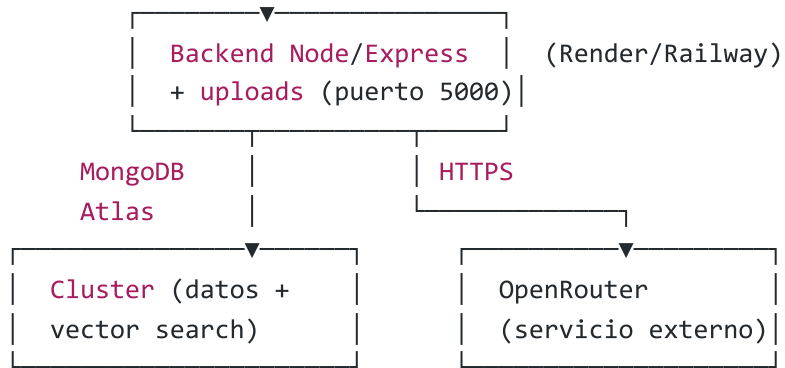
```

3. Diagrama de componentes



4. Diagrama de despliegue



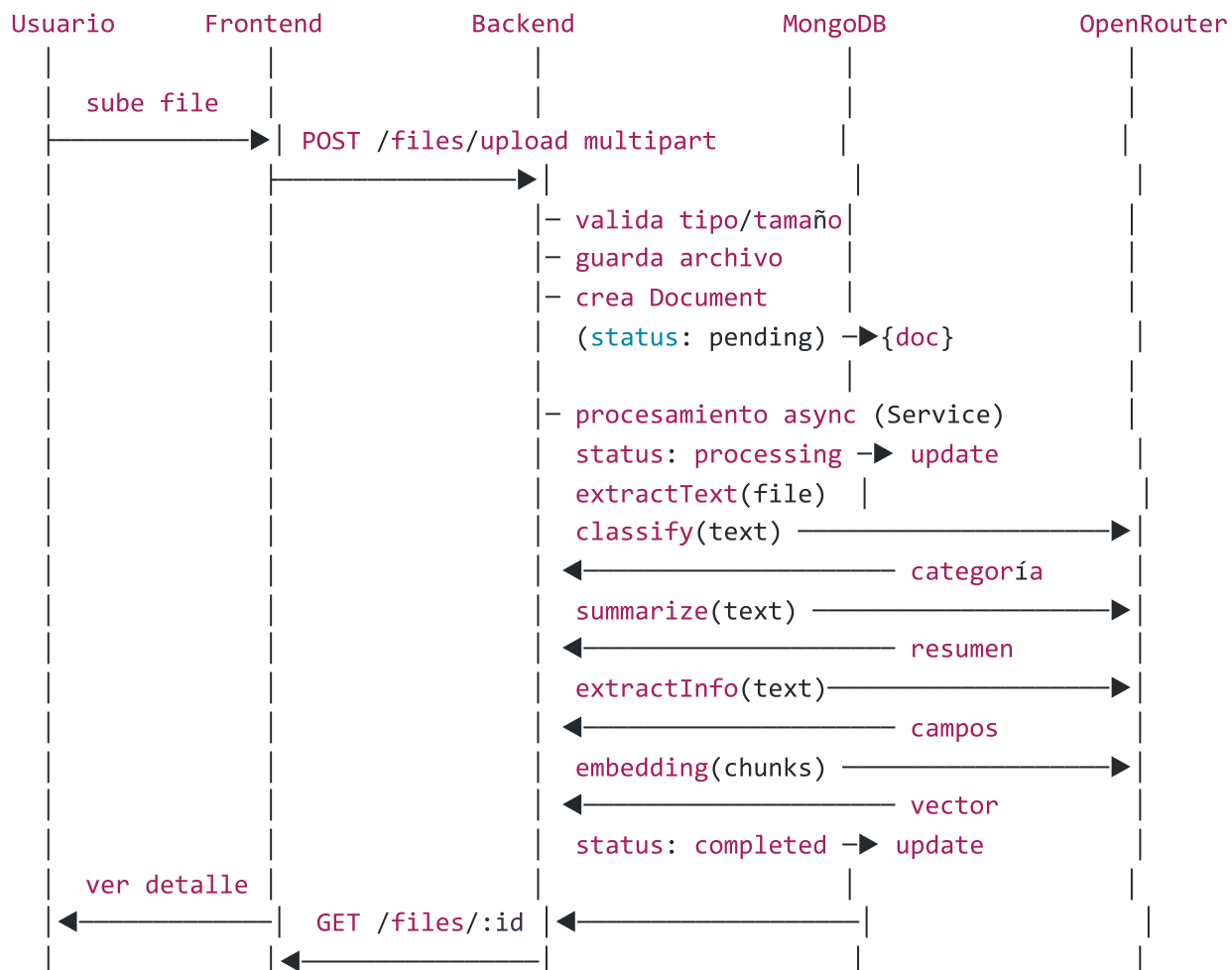


Alternativa local reproducible (si no da tiempo la nube)

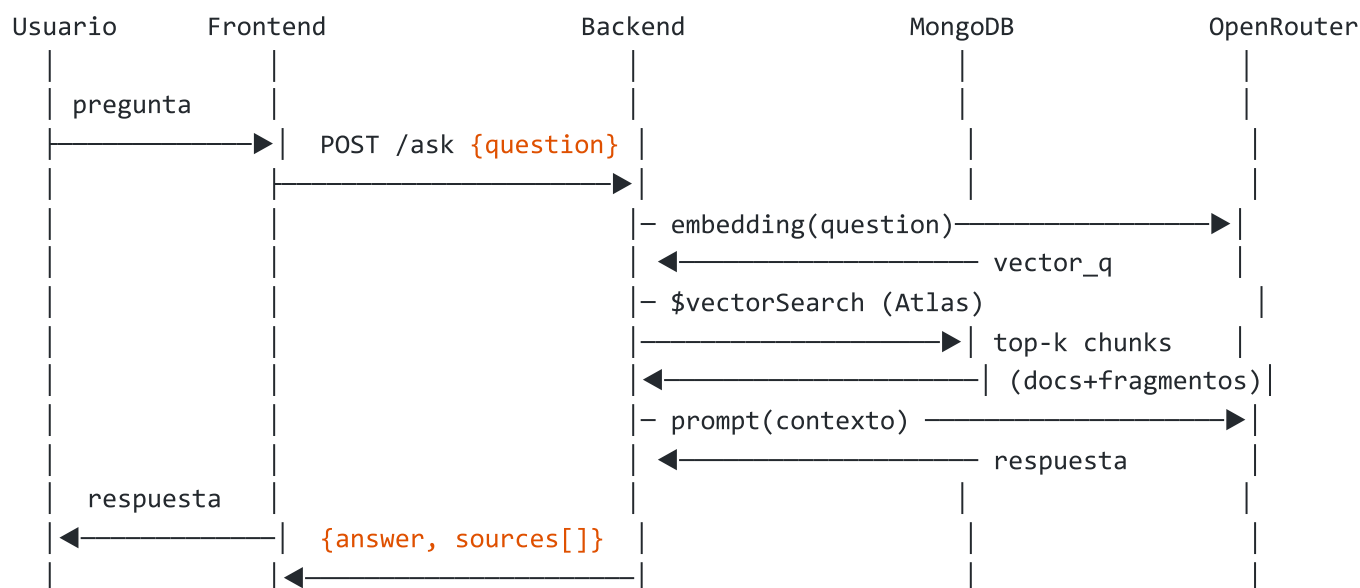
Despliegue en local: `git clone` → `npm install` (backend y frontend) → copiar `.env.example` a `.env` → `npm run dev`. La BD vive en Atlas (nube) y solo la API/UI corren en local.

5. Diagramas de secuencia

5.1 Carga y procesamiento de un documento



5.2 Consulta en lenguaje natural (RAG)



6. Modelo de datos

6.1 Entidades y relaciones

- User 1—N Repository (owner) · N—N (members)
- Repository 1—N Document (repository)
- User 1—N Document (owner)
- Document 1—N ProcessingLog (document)
- User 1—N ProcessingLog (user)

6.2 Colección users

Campo	Tipo	Restricciones	Descripción
_id	ObjectId	Auto	Identificador.
name	String	Obligatorio, 2-120	Nombre del usuario.
email	String	Obligatorio, único, formato email	Correo (minúsculas).
password	String	Obligatorio, ≥8, select:false	Hash bcrypt. Nunca se devuelve.
role	String	Enum: admin/editor/lector, def. editor	Rol.

Campo	Tipo	Restricciones	Descripción
company	String	≤150, opcional	Empresa.
isActive	Boolean	Def. true	Estado de la cuenta.
createdAt / updatedAt	Date	Timestamps	Auditoría.

6.3 Colección repositories

Campo	Tipo	Restricciones	Descripción
_id	ObjectId	Auto	Identificador.
name	String	Obligatorio, 1-150	Nombre del repositorio.
description	String	≤500	Descripción (def. "").
owner	ObjectId→User	Obligatorio	Propietario.
members	[ObjectId→User]	Def. []	Miembros con acceso de lectura/carga.
createdAt / updatedAt	Date	Timestamps	Auditoría.

6.4 Colección documents

Campo	Tipo	Restricciones	Desc
_id	ObjectId	Auto	Identif
filename	String	Obligatorio	Nomb físico e uploac
originalName	String	Obligatorio	Nomb origina archive
fileType	String	pdf/docx/txt	Tipo.
size	Number	Obligatorio	Tamañ bytes.
mimeType	String	Opcional	MIME archive
path	String	Obligatorio	Ruta fí

Campo	Tipo	Restricciones	Desc
repository	ObjectId→Repository	Obligatorio	Reposi
owner	ObjectId→User	Obligatorio	Subido
status	String	Enum pending/processing/completed/error, def. pending	Estado proces
category	String	Enum Contrato/Factura/Reporte/Otro, def. Otro	Clasific
summary	String	—	Resum
extractedInfo	Mixed	Def. {}	Campo IA.
textContent	String	—	Texto p extraíc búsque
chunks	[String]	—	Fragm texto (chunk
embedding	[Number]	—	Vector docum (longit model
processingError	String	—	Mensa error s
processedAt	Date	—	Fecha finaliza
createdAt / updatedAt	Date	Timestamps	Audito

Índices definidos:

- Text index sobre `filename`, `summary`, `textContent` (búsqueda por palabra clave).
- Índice compuesto (`repository`, `createdAt`) para listados rápidos.

6.5 Colección `processinglogs`

Campo	Tipo	Restricciones
_id	ObjectId	Auto
document	ObjectId→Document	Obligatorio
user	ObjectId→User	Obligatorio
status	String	Enum, def. pending
stage	String	extraccion/clasificacion/resumen/extraccion_inf
message	String	—
startedAt / finishedAt	Date	—
createdAt / updatedAt	Date	Timestamps

7. Diseño de la API (REST)

Autenticación

Método	Ruta	Cuerpo	Descripción
POST	/api/auth/register	{name,email,password,role?,company?}	Registro → token + usuario.
POST	/api/auth/login	{email,password}	Login → token + usuario.
GET	/api/auth/profile	— (JWT)	Perfil del usuario autenticado.

Repositorios (requieren JWT)

Método	Ruta	Cuerpo	Descripción
POST	/api/repos	{name, description}	Crear repositorio.

Método	Ruta	Cuerpo	Descripción
GET	/api/repos	—	Listar repos del usuario (owner/member).
GET	/api/repos/:id	—	Detalle + conteo de documentos.
PUT	/api/repos/:id	{name?, description?}	Editar (solo owner).
DELETE	/api/repos/:id	—	Eliminar repositorio y sus documentos (solo owner).

Archivos (requieren JWT)

Método	Ruta	Cuerpo	Descripción
POST	/api/files/upload	multipart: file , repositoryId	Subir archivo.
GET	/api/files? repositoryId=&status=	—	Listar/filtrar documentos.
GET	/api/files/:id	—	Detalle del documento (incluye análisis IA).
GET	/api/files/:id/download	—	Descargar archivo físico.
DELETE	/api/files/:id	—	Eliminar (owner o admin).

Procesamiento IA (requieren JWT)

Método	Ruta	Cuerpo	Descripción
POST	/api/files/:id/process	—	Ejecutar/repEjecutar procesamiento IA.
POST	/api/files/:id/ask	{question}	Pregunta sobre un documento específico (RAG).
POST	/api/ask	{question, repositoryId?}	Pregunta global (RAG) con fuentes.
GET	/api/search? q=&repositoryId=	—	Búsqueda por palabra clave + semántica.
GET	/api/dashboard	—	Indicadores del dashboard.

Nota: `/api/ask`, `/api/search` y `/api/dashboard` se implementan en el día 4-5; el diseño queda definido aquí.

8. Diseño de la integración de IA

8.1 Modelos usados

Tarea	Modelo	Justificación
Clasificación, resumen, extracción, RAG	deepseek/deepseek-chat	Modelo de chat de DeepSeek servido por OpenRouter; salida JSON estricta; bajo costo.
Embeddings	openai/text-embedding-3-small	1536 dimensiones, costo bajo, buena calidad para RAG (servido por OpenRouter).

8.2 Prompts (español)

- **Clasificación:** "Clasifica el siguiente documento en exactamente una de estas categorías: Contrato, Factura, Reporte, Otro. Responde en JSON: `{"categoria": "..."} .`"
- **Resumen:** "Resume el siguiente documento en 3-5 líneas en español. Devuelve solo el resumen."
- **Extracción:** según categoría. Ej. Factura: `{"fecha", "monto_total", "proveedor", "numero_factura"}` . Contrato: `{"partes", "fecha_de_firma", "vigencia", "objeto"}` . Se le pide JSON estricto.
- **RAG:** "Responde con base únicamente en el siguiente contexto extraído de documentos. Si no hay información, indícalo. Cita los fragmentos utilizados."

8.3 Cómo se define el prompt de RAG

1. El usuario escribe la pregunta.
2. Se genera `embedding(question)` .
3. Atlas Vector Search devuelve los top-k fragmentos más similares.
4. Se arma prompt: Contexto: ... \n Pregunta:
5. `deepseek/deepseek-chat` responde con base en el contexto e indicamos los documentos fuente.

8.4 Justificación de RAG + embeddings sobre BD vectorial aparte

Ver sección 1. Ventajas: un solo motor de datos (Atlas), cero infraestructura adicional, menor costo y administración, y es escalable al mismo motor de búsqueda.

9. Decisiones tecnológicas y justificación

Decisión	Alternativa	Justificación
MERN (Mongo/Express/React/Node)	Django+SQL, .NET	Conocido por el equipo; un solo lenguaje (JS) en todo el stack; JSON nativo alineado con Mongo.
JWT	Sesiones en servidor	Sin estado, fácil de usar en SPA, expiración configurable.
bcrypt	plain/SHA	Hash con sal recomendado para contraseñas.
MongoDB Atlas Vector Search	Pinecone/Qdrant local	Sin infraestructura extra, vector en la BD de datos.
Multer	formidable / carga directa a S3	Simple, maduro, validación de extensión/MIME y tamaño.
MongoDB (NoSQL)	PostgreSQL relacional	Flexibilidad de esquema (extractedInfo es variable según tipo), integración nativa con vectores.

10. Diseño básico de seguridad

- 1. **Contraseñas:** hash con bcrypt (10 rondas de sal) en `User.pre('save')` .
- 2. **Tokens:** JWT firmado con secreto desde variable de entorno, expiración `JWT_EXPIRES_IN` (7 días por defecto). Middleware `protect` valida el `Authorization: Bearer` .
- 3. **Roles:** middleware `authorize(roles...)` restringe acciones sensibles (admin/owner).
- 4. **Variables de entorno:** `.env` fuera del repo (ver `.gitignore`); `.env.example` documenta las variables requeridas. Clave de OpenRouter (`OPENAI_API_KEY`) y URI de Atlas nunca versionadas.
- 5. **Validación de archivos:** extensión + MIME permitidos (PDF/DOCX/TXT) y límite de tamaño (10 MB). Nombre de almacenamiento generado por el servidor (timestamp+random), evitando path traversal y colisiones.
- 6. **Errores:** manejador centralizado; en desarrollo se expone el stack, en producción no.
- 7. **Acceso a recursos:** cada consulta valida que el usuario sea owner o miembro del repositorio antes de devolver el documento.

Fin del documento 02.